

Deep Learning Supervisionato

NECKER

2 Marzo 2026

Indice

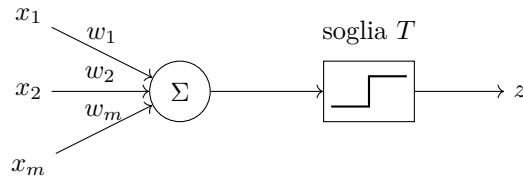
1	Perceptron	3
1.1	Struttura e Scopo	3
1.2	Vettori e Rappresentazione Grafica	3
1.3	Adaline e Madaline	4
1.4	Funzioni di Attivazione e Loss	5
1.4.1	Calcolo dell'Errore e Funzione di Loss	5
1.5	Minimo Locale vs Minimo Globale	5
1.6	Entropia e Divergenza KL	6
2	back propagation	9
2.1	con soft max	11
3	momentum method	13
4	nesterov momentum	13
5	batch	14
6	convoluzione discreta	14
6.1	stride, padding e pooling	15
6.2	percorso immagine	16
7	depth e vanishing gradient	16
7.1	vanishing gradient:	16
8	dying neurons e exploding gradient	17
8.1	dying neurons:	17
8.2	exploding gradient:	17
9	skip connection (res net)	18
10	Possibili problemi che possono sorgere:	18
10.1	riduzione canali	18
10.2	problema rotazioni	19
10.3	reti che svolgono più compiti	19
11	cnn vs fcnn e u-net	19
11.1	u-net	19
11.1.1	interpolation	20
11.1.2	transposed convolution / up convolution	20
11.2	allenamento u-net	20
12	rnn (recurrent neural network)	21
12.1	struttura:	21
12.2	Architettura e Logica delle RNN Classiche	21
12.3	bptt: (back propagation through time)	22

13 vanishing gradient e exploding gradient	22
14 lstm: (long short-term memory)	23
14.1 forget gate: (quanta memoria dimenticare)	23
14.2 input gate: (quanta informazione conservare)	24
14.3 output gate:	25
15 gru	26
15.1 reset gate:	26
15.2 update gate:	27
15.3 output:	27
16 bidirectional rnn	27
17 seq2seq con attenzione	28
17.1 struttura:	28
17.2 passaggi:	28
17.3 Back propagation:	28

1 Perceptron

1.1 Struttura e Scopo

L'obiettivo è quello di separare elementi di classi diverse all'interno dello spazio degli input tramite un confine lineare (una retta). Poiché il Perceptron calcola una somma ponderata, esso può solo dividere lo spazio in due semipiani netti: i punti in cui la combinazione lineare è positiva appartengono a una classe, gli altri alla classe opposta. Di conseguenza, il modello presuppone che i dati siano **linearmente separabili**; se le due classi fossero mischiate tra loro, una semplice retta non riuscirebbe a dividerle senza commettere errori.



Funzione di attivazione:

$$\phi(z(x)) = \begin{cases} 1 & \text{if } z(x) \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

$$z(x) = \sum x_i \cdot w_i - T (+b)$$

Dove T è la soglia (fissa) e $+b$ è il bias (varia, nel perceptron normale).

Aggiornamento:

$$w_i = w_i - \mu(F(x) - \phi(z(x))) \cdot x_i$$
$$b = b - \mu(F(x) - \phi(z(x)))$$

- μ = learning rate
- $F(x)$ = valore atteso (etichetta)
- $\phi(z(x))$ = previsione (0 neg, 1 pos)

Nota: in base a quanto l'input è grande, viene influenzata la correzione.

se x_i è grande, allora molto probabilmente è lui la causa dell'errore, quindi moltiplicarlo all'errore permette di pesare l'errore di conseguenza (se X_i è vicino a 0 allora molto probabilmente non è lui la causa dell'errore)

1.2 Vettori e Rappresentazione Grafica

Per semplicità ragioneremo a 2 dimensioni per capire come funziona il **Perceptron** ma nella realtà, funziona con lo stesso meccanismo a più dimensioni:

- $W = \{w_1, w_2\}$: vettore dei pesi
- $X = \{x_1, x_2\}$: vettore dei valori
- $W^T = \begin{Bmatrix} w_1 \\ w_2 \end{Bmatrix}$: vettore trasposto (graficamente non cambia nulla ma è manipolabile ora)

$$W^T X = \sum_{i=0}^n w_i x_i$$

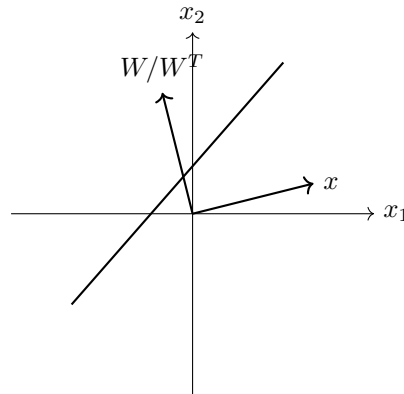
$$w_0 x_0 + w_1 x_1 + b = W^T X \implies \begin{cases} \text{se } \geq 0 = 1 \\ \text{se } < 0 = 0 \end{cases}$$

troviamo il coeff angolare:

$$w_1 x_1 = -w_0 x_0 - b$$

$$x_1 = -\left(\frac{w_0}{w_1}\right) x_0 - \left(\frac{b}{w_1}\right)$$

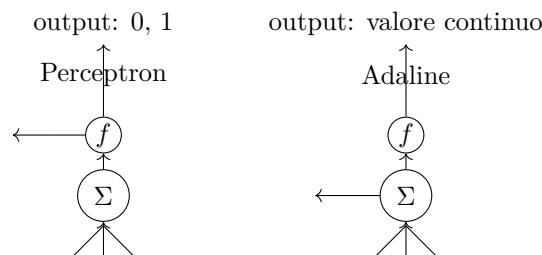
Ricavato $m = -\frac{w_0}{w_1}$: (dove il primo termine tra parentesi rappresenta m e il secondo q), possiamo disegnare il grafico della nostra retta:



Se $\mu = 1$, allora la correzione sarà la retta spostata per fittare quel punto.

1.3 Adaline e Madaline

Adaline: evoluzione del perceptron (prima o zero o 1, ora abbiamo una soglia variabile).



La differenza è che Adaline non tratta gli output in maniera uguale, invece di calcolare l'errore tra etichetta e 0 o 1 (dopo aver passato il valore continuo nella funzione a gradino), lo calcola col valore continuo stesso correggendo di più se lontano dalla soglia e di meno se vicino (la soglia rimane uguale).

correzione del peso (stessa di prima):

$$\Delta w_i = \eta \cdot (\text{Errore}) \cdot x_i$$

Madaline (Multi-Adaline): In un multiplo si aggiorna solo l'Adaline con i pesi che minimizzano l'errore (greedy). Lo strato finale (che di solito è una porta logica fissa come un AND o una regola di maggioranza) ti dice: "Per correggere l'errore e indovinare la risposta, basterebbe che almeno uno dei neuroni Adaline dello strato nascosto invertisse il proprio output da -1 a $+1$ ".

1.4 Funzioni di Attivazione e Loss

Le funzioni di attivazione introducono non-linearità nel modello, permettendo alla rete neurale di apprendere relazioni complesse. Dato l'input totale pesato del neurone z , definito come:

$$z = \sum_{i=1}^n w_i x_i + b \quad (1)$$

dove w_i sono i pesi, x_i le feature di input e b il bias, l'output (o attivazione) del neurone è dato da $a = F(z)$.

I principali tipi di funzioni di attivazione includono:

- **Funzione a scalino (Heaviside):** Utilizzata nel Perceptron originario, non è derivabile in $z = 0$ e ha derivata nulla altrove.
- **Sigmoide (Logistica):** Schiaccia l'output nell'intervallo $(0, 1)$, interpretabile come probabilità:

$$F(z) = \sigma(z) = \frac{1}{1 + e^{-z}} \quad (2)$$

- **ReLU (Rectified Linear Unit):** Definita come il massimo tra zero e il valore di input:

$$F(z) = \max(0, z) \quad (3)$$

- **Softplus:** Una variante smooth e differenziabile della ReLU (spesso confusa con essa):

$$F(z) = \log(1 + e^z) \quad (4)$$

1.4.1 Calcolo dell'Errore e Funzione di Loss

Per valutare la discrepanza tra l'output predetto dal neurone $a = F(x)$ e il valore target atteso $\phi(z(x))$, si utilizza una funzione di costo (Loss Function). Nel caso dell'errore quadratico medio (MSE) per un singolo pattern, la loss è definita come:

$$\text{ERR} = \frac{1}{2} (F(x) - \phi(z(x)))^2 = \frac{1}{2} (f(z(x)) - \phi(z(x)))^2 \quad (5)$$

oppure la versione non compatta:

$$\text{ERR} = \frac{1}{2} \left(y - \frac{1}{1 + e^{-(\sum_{i=1}^n w_i x_i + b)}} \right)^2 \quad (6)$$

L'obiettivo dell'addestramento è minimizzare E aggiornando i pesi nella direzione opposta al gradiente.

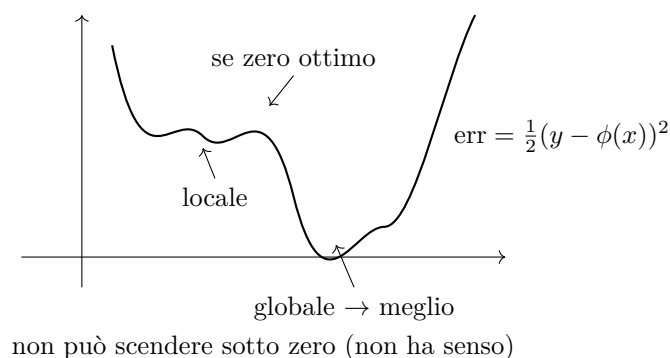
ATTENZIONE: generalmente si usa l'**MSE** poiché permette di correggere di più errori più grandi e meno errori più piccoli

Trovato l'errore, si deriva parzialmente rispetto ad ogni peso per trovare di quanto diminuisce.

passaggio spiegato bene nel capitolo della **Back Propagation**

1.5 Minimo Locale vs Minimo Globale

Nel gradiente (derivata della loss), quando ti trovi sul fianco di una duna, significa che cambiando il valore del peso w_1 , l'errore scende drasticamente. La combinazione lineare dentro la sommatoria $\sum w_i x_i + b$ sta modificando l'input della sigmoide in modo tale che l'output della rete si avvicina al target y . L'algoritmo (Backpropagation) calcola la pendenza di questa duna per capire se deve aumentare o diminuire il valore di w_1 .



Se il gradiente è zero (piatta) potremmo essere in un minimo locale e non globale.

1.6 Entropia e Divergenza KL

L'Entropia e Divergenza KL vengono introdotte per risolvere i limiti dell'**MSE**. Per comprendere l'inefficacia dell'errore quadratico medio nei task di classificazione associati a funzioni di attivazione sigmoidali, analizziamo utilizzando l'MSE.

Ipotizziamo nuovamente che il target reale sia $y = 1$ e che il modello commetta un errore macroscopico, generando un output fortemente errato pari a $a = 0.0001$ (il neurone è quasi spento, ma dovrebbe essere totalmente attivo). Supponiamo un input $x_i = 1$ e un tasso di apprendimento standard $\eta = 0.1$.

La formula della derivata parziale dell'MSE (il gradiente) rispetto al peso w_i include il termine di derivata della sigmoide $a(1 - a)$ dovuto alla regola della catena:

$$\text{Gradiente}_{MSE} = \frac{\partial \text{ERR}}{\partial w_i} = -(y - a) \cdot a(1 - a) \cdot x_i$$

Sostituendo i valori numerici all'interno dei singoli componenti:

- **Errore residuo:** $(y - a) = (1 - 0.0001) = 0.9999$ (L'errore puro è quasi massimo)
- **Derivata della sigmoide (Saturazione):** $a(1 - a) = 0.0001 \cdot (1 - 0.0001) = \mathbf{0.00009999}$

Moltiplicando i termini per ottenere il gradiente complessivo:

$$\text{Gradiente}_{MSE} = -0.9999 \cdot 0.00009999 \cdot 1 \approx \mathbf{-0.00009998}$$

Calcoliamo ora la variazione effettiva applicata al peso (Δw_i) tramite la regola del passo del gradiente:

$$\Delta w_i = -\eta \cdot \text{Gradiente}_{MSE} = -0.1 \cdot (-0.00009998) = \mathbf{+0.00009998}$$

Considerazione Geometrica: Nonostante l'errore del modello sia immenso (prossimo a 1), il passo di aggiornamento del peso è infinitesimale (nell'ordine di 10^{-5}). Questo accade perché il termine $a(1 - a)$ agisce come un collo di bottiglia che azzerla la pendenza. Di conseguenza, la rete si trova bloccata in un **altopiano piatto (plateau)** indotto dalla saturazione della sigmoide, rendendo l'addestramento virtualmente immobile a prescindere dall'entità dell'errore.

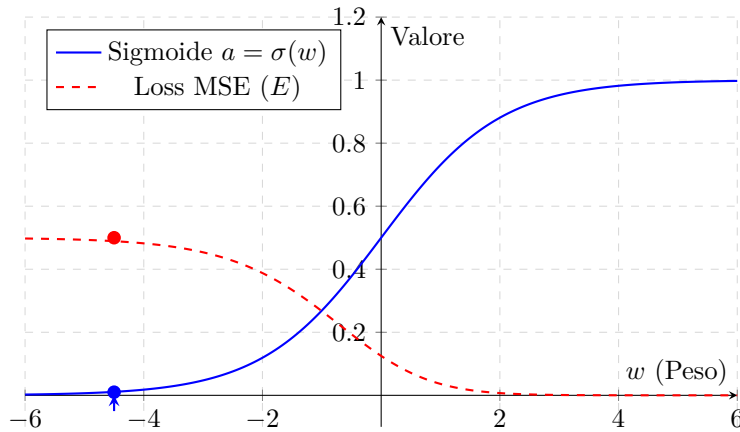


Figura 1: Visualizzazione della saturazione del gradiente con la funzione di costo MSE per un target $y = 1$. Quando il peso spinge il neurone nella zona errata ($w \rightarrow -\infty$), la Loss si stabilizza sul suo massimo ma perde totalmente la pendenza.

KL (Kullback-Leibler):

$$KL = \sum_i P(i) \log \left(\frac{P(i)}{Q(i)} \right)$$

- Se $P(i) < Q(i) \rightarrow$ allora log negativo
- $P(i)$ = probabilità vera (output della rete visto come probabilità)
- $Q(i)$ = probabilità stimata (etichetta dei dati di training)

Sommatoria di tutte le differenze di probabilità, utilizzata come funzione di loss (sempre positiva). La disuguaglianza per il teorema di Jensen: anche se alcuni valori sono negativi, la somma totale sarà positiva.

Cross Entropy:

$$curvaH(P, Q) = - \sum_i P(i) \log Q(i)$$

Anche lei misura quanto una probabilità sia lontana dalla realtà. (Il segno meno rende sempre positivo).

Collegamento:

$$KL = H(P, Q) - H(P)$$

Dove $H(P)$ è l'entropia di P (costante).

Binary Cross Entropy: (Usata come loss function per i problemi di classificazione binaria).

- $y_i \in (0, 1)$ = probabilità prevista dal modello
- $d_i \in (0, 1)$ = etichetta

$$BCE = -d_i \log(y_i) - (1 - d_i) \log(1 - y_i)$$

Penalizza gli errori molto di più dell'errore quadratico medio.

Esempio Numerico Applicato:

Immaginiamo un problema di classificazione binaria in cui l'etichetta reale è la Classe 1. Le due distribuzioni di probabilità sono:

- **Realtà (P):** $P(1) = 1$ e $P(0) = 0$ (Distribuzione certa, target fisso)
- **Modello (Q):** $Q(1) = y$ e $Q(0) = 1 - y$ (Output del neurone)

Calcoliamo l'Entropia della realtà $H(P)$ per verificare che sia una costante:

$$H(P) = - \sum_i P(i) \log P(i) = -(1 \cdot \log(1) + 0 \cdot \log(0)) = 0$$

Dato che $H(P) = 0$, dalla relazione $KL = H(P, Q) - H(P)$ ne consegue che in questo scenario ideale $KL = H(P, Q)$.

Analizziamo numericamente il comportamento della Loss in due scenari opposti:

1. **Scenario A (Il modello indovina):** La rete è sicura al 90% della risposta corretta $\rightarrow y = 0.9$.

- **Cross-Entropy:** $H(P, Q) = -(1 \cdot \log(0.9) + 0 \cdot \log(0.1)) = -(-0.105) = \mathbf{0.105}$
- **Divergenza KL:** $KL = \sum_i P(i) \log \left(\frac{P(i)}{Q(i)} \right) = 1 \cdot \log \left(\frac{1}{0.9} \right) + 0 = \mathbf{0.105}$

Risultato: Entrambe le metriche restituiscono un valore molto basso (vicino a zero), indicando che le due distribuzioni sono quasi sovrapposte.

2. **Scenario B (Il modello sbaglia di grosso):** La rete assegna solo il 10% alla risposta corretta $\rightarrow y = 0.1$.

- **Cross-Entropy:** $H(P, Q) = -(1 \cdot \log(0.1) + 0 \cdot \log(0.9)) = -(-2.302) = \mathbf{2.302}$
- **Divergenza KL:** $KL = 1 \cdot \log \left(\frac{1}{0.1} \right) + 0 = \mathbf{2.302}$

Risultato: La perdita informativa e il costo aumentano drasticamente (da 0.105 a 2.302), generando un forte gradiente che spingerà i pesi a correggersi rapidamente durante la Backpropagation.

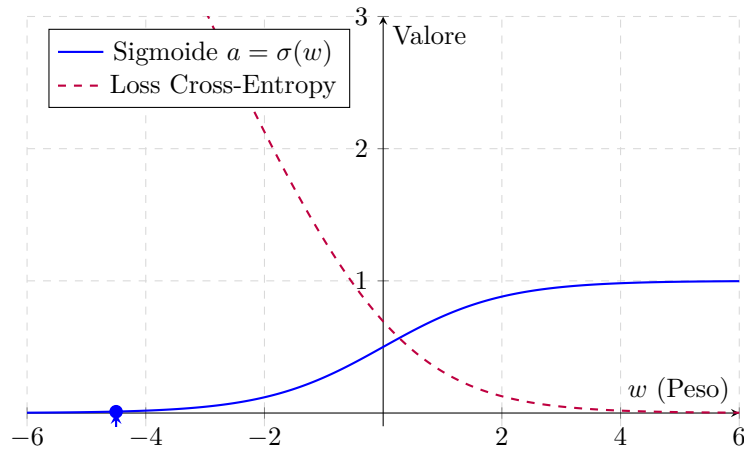


Figura 2: Visualizzazione della Loss con la funzione di costo Cross-Entropy per un target $y = 1$. Quando il peso spinge il neurone nella zona errata ($w \rightarrow -\infty$), la curva non si appiattisce come nell'MSE, ma impenna verso l'infinito garantendo un gradiente forte e costante.

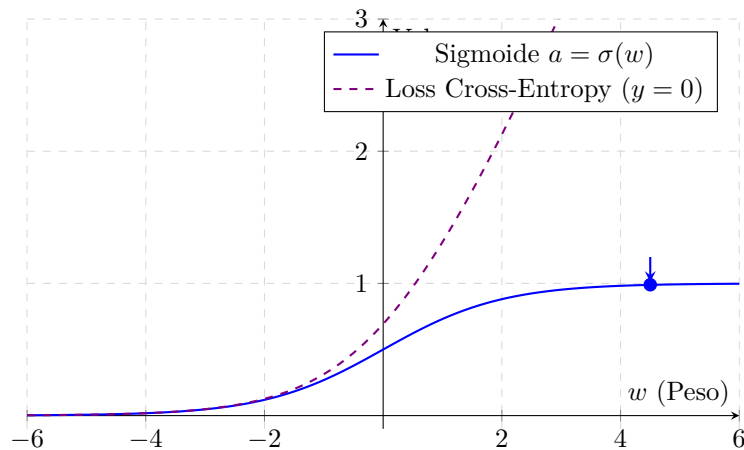
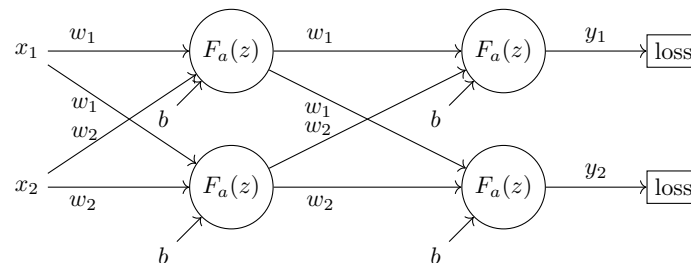


Figura 3: Visualizzazione della Cross-Entropy per un target $y = 0$. Se il peso cresce verso destra ($w \rightarrow +\infty$), l'output del neurone tende a 1 (errore massimo). La funzione di costo risponde con una pendenza ripida e speculare, forzando l'ottimizzatore a invertire la rotta.

2 back propagation

La back propagation è la strategia utilizzata per applicare la correzione dei pesi di ogni neurone. Fino ad ora abbiamo visto come funziona un singolo neurone, nella realtà per effettuare una previsione usiamo più neuroni alla volta, in parallelo e su più strati. Poiché avremo una singola previsione calcolata come output finale dell'ultimo strato, calcoleremo la **Loss** solo lì per poi propagare l'errore all'indietro negli altri strati.



PROCEDIMENTO:

strato finale:

- calcolo per tutti i neuroni della rete: $z = w_1x_1 + w_2x_2 + b$
- e poi: $F_a(z) = \text{ReLU}(z)$ la nostra y
- $\text{loss}(F_a(z)) = \frac{1}{2}(\hat{y} - y)^2$ (posso farlo solo per l'ultimo strato)

quindi:

$$\text{Loss} = \underbrace{\frac{1}{2} \left(\hat{y} - \overbrace{\text{ReLU}(\underbrace{w_1x_1 + w_2x_2 + b}_z)}^y \right)}_{\text{Loss}}$$

derivo:

- la loss rispetto $F_a(z) \rightarrow \frac{\partial \text{loss}}{\partial y}$
- la y rispetto a $z \rightarrow \frac{\partial y}{\partial z}$
- la z rispetto a $w_1, w_2, b \rightarrow \frac{\partial z}{\partial w_1}, \frac{\partial z}{\partial w_2}, \frac{\partial z}{\partial b}$

chain rule: per trovare quanto influisce un singolo peso o bias devo moltiplicare le derivate rispetto alla funzione composta

$$\frac{\partial \text{loss}}{\partial w} = \frac{\partial \text{loss}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w}$$

grazie alla chain rule, trovo le variazioni e aggiorno i pesi.

dei pesi e bias:

$$\begin{aligned} \Delta w_1 &= \frac{\partial \text{loss}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_1} \\ \Delta w_2 &= \frac{\partial \text{loss}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_2} \\ \Delta b &= \frac{\partial \text{loss}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial b} \end{aligned}$$

aggiornamento pesi e bias:

$$\begin{aligned} w_1 &= w_1 - \mu \cdot \Delta w_1 \\ w_2 &= w_2 - \mu \cdot \Delta w_2 \\ b &= b - \mu \cdot \Delta b \end{aligned}$$

(μ = learning rate)

neurone precedente:

A questo punto l'unica cosa che manca è la derivata della funzione di loss dato che non avevamo $\hat{y}$. quindi ora che abbiamo :

$$\frac{\partial \text{loss}}{\partial y} = \sum_{i=1}^m w_i \frac{\partial \text{loss}}{\partial y_i}$$

ad esempio:

$$\left(w_1 \frac{\partial \text{loss}}{\partial y_1} + w_2 \frac{\partial \text{loss}}{\partial y_2} \right)$$

ATTENZIONE: i pesi sono quelli non aggiornati

- Per poi calcolare come prima: $\Delta w_1 = \frac{\partial \text{loss}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_1}$, $\Delta w_2 = \frac{\partial \text{loss}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_2}$, $\Delta b = \frac{\partial \text{loss}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial b}$ e poi aggiornare i pesi.

paragone con:

$$w = w - \mu(F(y) - \phi(z)) \cdot x \quad \text{vs} \quad w = w - \mu \left(\frac{\partial \text{loss}}{\partial w} \right)$$

$$b = b - \mu(F(y) - \phi(z)) \quad \text{vs} \quad b = b - \mu \left(\frac{\partial \text{loss}}{\partial b} \right)$$

$$\frac{\partial \text{loss}}{\partial w} = \frac{\partial \text{loss}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w} \quad (\text{la } x \text{ è già considerata nella derivata di } z \text{ rispetto a } w)$$

$$\frac{\partial \text{loss}}{\partial b} = \frac{\partial \text{loss}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial b} \quad (\text{la derivata di } z \text{ rispetto a } b \text{ è sempre } = 1 \text{ quindi } x \text{ non si considera})$$

Dimostrazione Analitica delle Derivate rispetto a z : Data la combinazione lineare del neurone: $z = w_1 x_1 + w_2 x_2 + b$

Quando si applica la derivazione parziale, tutte le variabili diverse dalla variabile di derivazione vengono trattate come costanti matematiche (la cui derivata è nulla):

- **Derivata rispetto al peso w_1 :** Il peso è legato moltiplicativamente all'input x_1 . La componente $(w_2 x_2 + b)$ decade a 0:

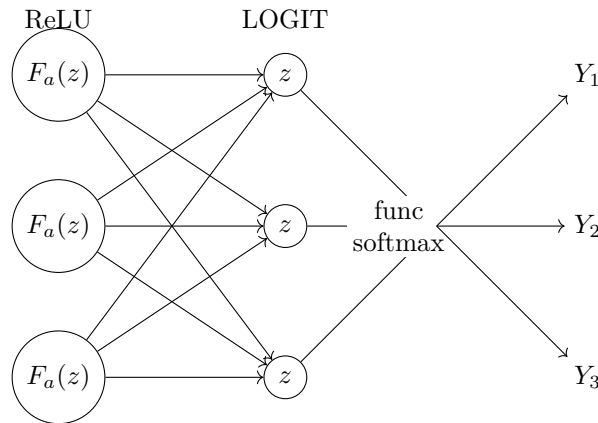
$$\frac{\partial z}{\partial w_1} = \frac{\partial}{\partial w_1}(w_1 x_1) + 0 = x_1$$

- **Derivata rispetto al bias b :** Il bias è un termine noto additivo isolato e indipendente dagli input. La componente $(w_1 x_1 + w_2 x_2)$ decade a 0, mentre la derivata di una variabile rispetto a se stessa è sempre unitaria:

$$\frac{\partial z}{\partial b} = 0 + \frac{\partial}{\partial b}(b) = 1$$

Conseguenza geometrica: Nei rispettivi passi del gradiente (Δw e Δb), l'input x compare come fattore di scala solo per i pesi (poiché ne determina la responsabilità nell'errore), mentre scompare per il bias, il cui aggiornamento dipende unicamente dall'errore cumulato a valle del neurone.

2.1 con soft max



Soft Max:

$$\hat{Y}_i = \frac{\exp(z_i)}{\sum_{j=1}^c \exp(z_j)}$$

- $\exp(z_i)$ → esponenziale permette di rendere più evidenti le diff. tra i vari z
- $\sum_{j=1}^c \exp(z_j)$ → in input prende tutti gli z
- $\hat{Y}_i$ → stessa cosa di $\frac{z}{\text{tot } z}$ = formula probabilità classica

- $c = n^\circ$ classi
- Y_i = label
- $\hat{Y}_i$ = previsione

calcolo della loss:

per effettuare la **Back Propagation**, nella soft max il discorso cambia radicalmente.

- **ReLU:** si passa direttamente z direttamente nella sigmoide per ottenere y (y dipende solo da un singolo z)
- **Soft Max:** per creare y , si prendono tutte le z risultanti di tutti i neuroni e si mettono nella funzione soft max la quale restituirà la probabilità di ogni classe (ci saranno tante y tante quante sono le z dato che tutte devono stare una volta al numeratore)

detto ciò dobbiamo introdurre 2 concetti fondamentali:

- in softmax la loss è la cross entropy $\left(-\sum_{i=1}^c Y_i \log(\hat{Y}_i)\right)$
- in softmax la label è un vettore con tutti zero e un 1 solo per la classe corretta $([0, 0, 0, 0, 1, 0, 0, 0, 0])$

quindi la loss sarà: $-(0 \cdot \log(\hat{Y}_1) + 0 \cdot \log(\hat{Y}_2) + 1 \cdot \log(\hat{Y}_s) \dots)$

cioè:

$$\text{loss} = -\log(\hat{Y}_i) \implies \frac{\partial \text{loss}}{\partial \hat{Y}_i} = -\frac{y_i}{\hat{Y}_i} = -\frac{1}{\hat{Y}_i}$$

(nota: $\frac{d}{dx} \log x = \frac{1}{x}$)

come calcolare i pesi di ciascun logit in funzione della loss:

poichè bisogna partire da y_i e tornare indietro, notiamo che per ogni y dobbiamo calcolare la derivata di softmax $\left(\frac{\partial \hat{Y}_i}{\partial z_i}\right)$ per ogni z_i :

in teoria si calcolerebbero tutte le derivate per ogni caso, cioè per ogni Y prevista, in funzione di tutte le z .
es con 3 z : $\frac{\partial \hat{Y}_1}{\partial z_1}, \frac{\partial \hat{Y}_1}{\partial z_2}, \frac{\partial \hat{Y}_1}{\partial z_3} \quad \frac{\partial \hat{Y}_2}{\partial z_1}, \frac{\partial \hat{Y}_2}{\partial z_2}, \frac{\partial \hat{Y}_2}{\partial z_3} \quad \frac{\partial \hat{Y}_3}{\partial z_1}, \frac{\partial \hat{Y}_3}{\partial z_2}, \frac{\partial \hat{Y}_3}{\partial z_3}$

e andrebbero poi sommate tutte per la regola delle derivate parziali, ottenendo 3 derivate (1 per ogni $\hat{Y}$).

MA sappiamo che la derivata di softmax generale è (rispetto a z_i):

caso in cui $\hat{Y}_i$ e $z_j \rightarrow i = j$ (z corrisponde al neurone stesso)

$$\frac{e^{z_i} \cdot (\sum_{k=1}^c e^{z_k}) - e^{z_i} \cdot e^{z_i}}{(\sum_{k=1}^c e^{z_k})^2} = \frac{e^{z_i} [(\sum_{k=1}^c e^{z_k}) - e^{z_i}]}{(\sum_{k=1}^c e^{z_k})^2} = \frac{e^{z_i}}{\sum_{k=1}^c e^{z_k}} \cdot \frac{\sum_{k=1}^c e^{z_k} - e^{z_i}}{\sum_{k=1}^c e^{z_k}}$$

$$= \hat{Y}_i \cdot \left(\frac{\sum_{k=1}^c e^{z_k}}{\sum_{k=1}^c e^{z_k}} - \frac{e^{z_i}}{\sum_{k=1}^c e^{z_k}} \right) = \hat{Y}_i \cdot (1 - \hat{Y}_i)$$

attenzione: per calcolare la derivata di softmax si usa la regola del quoziente: $dx = \frac{(\text{derivata num}) \cdot \text{den} - \text{num} \cdot (\text{derivata den})}{(\text{den})^2}$

caso in cui $\hat{Y}_i$ e $z_j \rightarrow i \neq j$ (z NON corrisponde al neurone stesso)

$$\frac{0 \cdot \sum_{k=1}^c e^{z_k} - e^{z_i} \cdot e^{z_j}}{(\sum_{k=1}^c e^{z_k})^2} = -\frac{e^{z_i}}{\sum_{k=1}^c e^{z_k}} \cdot \frac{e^{z_j}}{\sum_{k=1}^c e^{z_k}} = -\hat{Y}_i \cdot \hat{Y}_j$$

perché nel caso $i = j \rightarrow e^{z_i}$? e $i \neq j \rightarrow 0$? questo perché nei 2 casi il termine sopravvive solo se $i = j$.
dimostrazione:

- $i = j : \hat{Y}_i = \frac{e^{z_i}}{e^{z_i} + \sum_{k \neq i} e^{z_k}} \rightarrow \frac{\partial \hat{Y}_i}{\partial z_i} = \text{derivata di } \frac{e^{z_i}}{e^{z_i} + K}$ (K è una costante dato nelle derivate composte le altre variabili sono considerate costanti)
- $i \neq j : \hat{Y}_i = \frac{e^{z_i}}{e^{z_j} + \sum_{k \neq j} e^{z_k}} \rightarrow \frac{\partial \hat{Y}_i}{\partial z_j} = \text{derivata di } \frac{K}{e^{z_j} + K}$ (la derivata del num è zero)

quindi la derivata di softmax si suddivide in 2 casi:

- se $i = j \rightarrow \hat{Y}_i(1 - \hat{Y}_i)$
- se $i \neq j \rightarrow -\hat{Y}_i \hat{Y}_j$

per la regola della catena: $\frac{\partial \text{loss}}{\partial z_i} = \frac{\partial \text{loss}}{\partial \hat{Y}_i} \cdot \frac{\partial \hat{Y}_i}{\partial z_i}$
anche qui abbiamo 2 casi (ricordando che $\frac{\partial \text{loss}}{\partial \hat{Y}_s} = -\frac{1}{\hat{Y}_s}$):

- se $i = j \rightarrow \hat{Y}_i \cdot (1 - \hat{Y}_i) \cdot \left(-\frac{1}{\hat{Y}_i}\right) = -(1 - \hat{Y}_i) = \hat{Y}_i - 1$
- se $i \neq j \rightarrow -\hat{Y}_i \hat{Y}_j \cdot \left(-\frac{1}{\hat{Y}_i}\right) = \hat{Y}_j$

poiché la nostra i è l'etichetta corretta notiamo:

- per il caso corretto ($i = j$) il gradiente di discesa è: $\hat{Y}_i - 1$ (ciò significa che è o zero o negativo, quindi scende sempre)
- per il caso errato invece: $\hat{Y}_i$ (che è sempre positivo, quindi sempre in discesa di se stesso, è sbagliato della quantità che rappresenta)

infine calcoliamo $\frac{\partial z_i}{\partial w_j}$ e troviamo Δw con:

$$\frac{\partial \text{loss}}{\partial w_i} = \frac{\partial \text{loss}}{\partial z_i} \cdot \frac{\partial z_i}{\partial w_i}$$

3 momentum method

lo scopo è quello di aumentare la correzione se si sta andando sempre nella stessa direzione e diminuirla se la correzione oscilla (come una pallina che cade in una discesa).

formula aggiornamento classica:

$$w = w - \mu \Delta w$$

nuova:

$$\begin{aligned} v_t &= \beta v_{t-1} - \mu \Delta w_t \\ w_t &= w_{t-1} + v_t \end{aligned}$$

(v_t calcolata e inserita sotto)
 β = coefficiente di momentum

v_{t-1} rappresenta la velocità dell'aggiornamento precedente e v_0 è uguale a zero (il primo aggiornamento diventa quindi $w = w - \mu \Delta w$).

quindi:

- prima aggiorna la velocità basandosi sulla pos attuale
- poi aggiorna i pesi con la velocità calcolata

4 nesterov momentum

Nesterov è più preciso poiché, dato che la velocità è cumulativa, infatti:

1. calcola già la pos del peso futura con la velocità attuale
2. calcola la loss sul peso previsto per trovare la velocità (se fossimo velocissimi ma sul punto di risalire, ci permette di rallentare prima di vedere effettivamente che la loss risalta)
3. aggiorna il peso con la velocità ottima

variazione della formula per usarla:

$$\begin{aligned} \tilde{w} &= w_t + \beta v_{t-1} \quad (\beta = \text{coefficiente momentum}) \\ v_t &= \beta v_{t-1} - \mu \cdot \left(\frac{\partial \text{loss}}{\partial \tilde{w}} \right) \\ w_{t+1} &= w_t + v_t \end{aligned}$$

spiegazione:

- $\tilde{w}$ rappresenta la previsione del nuovo peso in funzione della v_{t-1} (come se aggiornassimo il peso senza fare la loss)
- calcoliamo v_t in funzione della nostra previsione (facciamo la loss sul peso $\tilde{w}$ per vedere quanto è distante dalla realtà la nostra previsione)
- aggiorniamo effettivamente il peso (w_{t+1}) con la nuova velocità

5 batch

Capito come si effettua l'aggiornamento, dobbiamo capire quando si effettua. Infatti aggiornare i pesi ad ogni singolo esempio del dataset spesso risulta inefficiente. Per questo motivo si dividono i dati in **batch**.

batch: un gruppo di dataset. es: $D = \left\{ \overbrace{(x_1, y_1), (x_2, y_2)}^{\text{batch 1}}, \overbrace{(x_3, y_3), (x_4, y_4)}^{\text{batch 2}} \right\}$

- **loss batch** = media della loss su tutti i train set del batch (usata per calcolare la loss tot)
- **loss tot** = loss media di tutti i batch di una singola epoca, usata come indicatore per vedere se la rete sta migliorando o meno

$$\text{loss tot} = \text{media delle loss batch}$$

- batch piccolo: + varianza tra i vari batch
- batch grosso: - varianza tra i vari batch

6 convoluzione discreta

La convoluzione discreta è il primo step che subisce un'immagine all'interno di una rete neurale, consiste nell'elaborare l'immagine in modo da accentuare i dettagli rilevanti (naturalmente dipende da come è stata addestrata la rete).

$$y = \sum x(a) \cdot h(n - a) = (x \cdot h) \cdot n$$

- $\sum x(a)$ indica che stiamo scorrendo su tutti i pixel dell'immagine ($x(a)$)
- $h(n - a)$ = filtro $\rightarrow$ indicato così per dire che si sposta nella matrice per tutti i pixel

ESEMPIO:

Considerando il calcolo del valore del pixel d'uscita nella posizione spaziale $n = 3$, l'equazione si formalizza come:

$$y(3) = \sum_a x(a) \cdot h(3 - a)$$

Sviluppando esplicitamente la sommatoria per l'intorno locale del pixel (ipotizzando ad esempio $a \in \{2, 3, 4\}$), si osserva analiticamente come l'operatore si focalizzi esclusivamente su quella specifica zona di input:

$$\text{Per } a = 2 \implies x(2) \cdot h(3 - 2) = x(2) \cdot h(1) \quad (\text{pixel precedente} \times \text{peso destro del filtro})$$

$$\text{Per } a = 3 \implies x(3) \cdot h(3 - 3) = x(3) \cdot h(0) \quad (\text{pixel centrale} \times \text{peso centrale del filtro})$$

$$\text{Per } a = 4 \implies x(4) \cdot h(3 - 4) = x(4) \cdot h(-1) \quad (\text{pixel successivo} \times \text{peso sinistro del filtro})$$

La somma algebrica di questi tre contributi determina il valore finale mappato in $y(3)$.

LOGICA:

- il filtro è una matrice particolare che serve a identificare una caratteristica particolare (omogeneità tra pixel per esempio)
- la matrice viene applicata a tutti i pixel circostanti e i valori sommati

es:

$$h = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

→ filtro di sobel (rileva i bordi). ogni posizione indica che il pixel vicino, dove lo zero centrale è il nostro pixel.

- il nuovo valore viene salvato in una copia dell'immagine
- risultato: pixel esaltati facendo notare le differenze

filtri 3×3 vs 9×9 :

- i 3×3 costano meno, infatti per ogni pixel 9 moltiplicazioni invece di 81
- i 3×3 sono più salienti (i 9×9 guardano un'area più ampia mitigando l'informazione)

6.1 stride, padding e pooling

1. **stride:** di quanti pixel si sposta il filtro.
2. **padding:** aggiungo pixel ai lati per evitare di scalare l'immagine (se ho un filtro 3×3 il primo pixel non lo vedrei).

$$\boxed{n_{\text{step}} = \frac{F-1}{2}} \quad F = \text{lunghezza matrice filtro}$$

3. **pooling:** riduzione della dimensionalità dell'immagine, è un filtro applicato all'immagine di 2 tipi:

- **max pooling:** $\begin{bmatrix} 1 & 3 \\ 4 & 8 \end{bmatrix} = 8$ (prende il valore massimo)

- **mean pooling:** $\begin{bmatrix} 1 & 3 \\ 4 & 8 \end{bmatrix} = 4$ (facendo la media tra tutti)

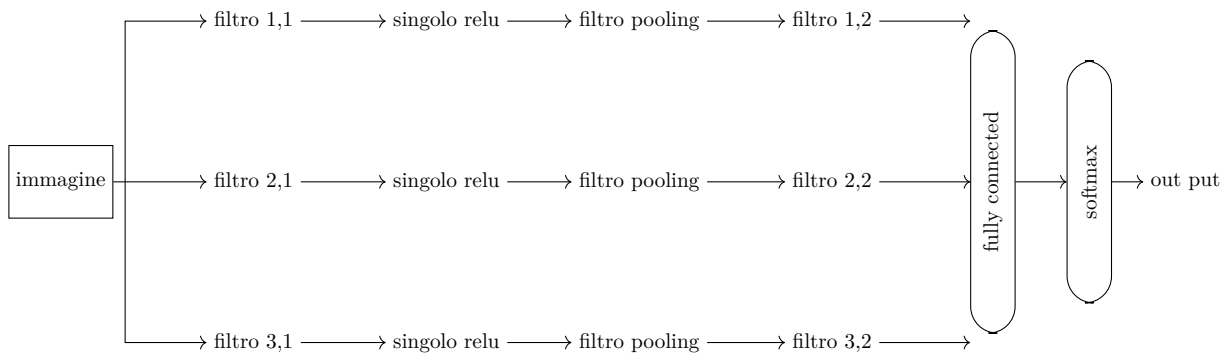
• filtri 2d hanno 3 dimensioni (immagini composte da: - immagini bianco e nero - immagini rgb), ma il loro output è in 2d (risultato dell'immagine).

$$\boxed{W = [D, D, I, F]} \quad \begin{array}{l} D = \text{dimensioni matrice (lung / larg)} \\ I = n^\circ \text{ strati immagine} \\ F = n^\circ \text{ filtri} \end{array}$$

W = tensore dei filtri 4d.

se si hanno 6 filtri si avranno come output 6 immagini 2d a cui si potranno applicare ancora altri filtri (slide 70 lezione 8).

6.2 percorso immagine



- filtri in parallelo: filtri di tipo diverso (es: linee, colore, punti).

- filtri in sequenza: filtri che evidenziano caratteristiche più specifiche (es: primo filtro → linee, secondo filtro → quadrati).
in pratica:

- il filtro evidenzia la caratteristica
- il relu la impara e la riconosce

filtri (pesi e bias):

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

questi 9 numeri sono anche i pesi. Inoltre, calcolato il nuovo valore del pixel si aggiunge il bias.
nella back propagation si tratta come se fosse un neurone e si aggiornano pesi e bias.

7 depth e vanishing gradient

La **depth** indica la profondità della rete. Più layer mettiamo, più l'errore aumenta. Questo per il fenomeno del vanishing gradient, il quale consiste nel concetto che più si va indietro più si aggiornano i pesi lentamente.

7.1 vanishing gradient:

ne soffrono le funzioni come la sigmoide, o la tanh (meno grave). Per capirlo basta guardare la derivata della sigmoide:

$$\underbrace{\frac{1}{1+e^{-x}} \cdot \left(1 - \frac{1}{1+e^{-x}}\right)}_{\text{sempre } \leq 0.25}$$

quindi andando indietro negli strati il gradiente diventa sempre più piccolo: se si vuole aggiornare un peso del primo strato (quello vicino all'input). Per calcolare il suo gradiente, la catena di moltiplicazioni deve attraversare tutti gli strati successivi per arrivare fino alla Loss:

$$\frac{\partial \text{Loss}}{\partial w_{\text{strato1}}} = \frac{\partial \text{Loss}}{\partial y_4} \cdot \underbrace{\frac{\partial y_4}{\partial z_4}}_{\text{Sigmoid 4}} \cdot \underbrace{\frac{\partial z_4}{\partial y_3}}_{\text{Sigmoid 3}} \cdot \underbrace{\frac{\partial y_3}{\partial z_3}}_{\text{Sigmoid 2}} \cdot \underbrace{\frac{\partial z_3}{\partial y_2}}_{\text{Sigmoid 1}} \cdot \frac{\partial y_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial y_1} \cdot \frac{\partial y_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{\text{strato1}}}$$

Anche se la Loss iniziale ($\frac{\partial \text{Loss}}{\partial y_4}$) è gigantesca (es. hai sbagliato completamente la previsione), guarda cosa succede quando passi attraverso i "freni" delle funzioni di attivazione. Nel caso peggiore/standard, ogni derivata della sigmoide vale al massimo 0.25. Sostituiamo i numeri nella catena isolando solo le attivazioni:

$$\text{Gradiente} \propto \text{Loss} \cdot 0.25 \cdot 0.25 \cdot 0.25 \cdot 0.25$$

$$\text{Gradiente} \propto \text{Loss} \cdot (0.25)^4$$

8 dying neurons e exploding gradient

8.1 dying neurons:

ne soffrono i neuroni di tipo **ReLU** poichè è una funzione definita a tratti che mappa l'input nel modo seguente:

$$f(z) = \max(0, z) = \begin{cases} z & \text{se } z \geq 0 \\ 0 & \text{se } z < 0 \end{cases} \quad (7)$$

La sua derivata prima rispetto all'input z (ovvero la pendenza della curva) corrisponde a una funzione a scalino:

$$f'(z) = \begin{cases} 1 & \text{se } z > 0 \\ 0 & \text{se } z < 0 \end{cases} \quad (8)$$

Nota ingegneristica: Poiché la derivata per valori positivi è costantemente pari a 1, la ReLU non fa da "freno" o collo di bottiglia durante la Backpropagation. Il gradiente passa intatto senza subire lo schiacciamento esponenziale tipico della Sigmoide.

Grafico della Funzione ReLU e della sua Derivata

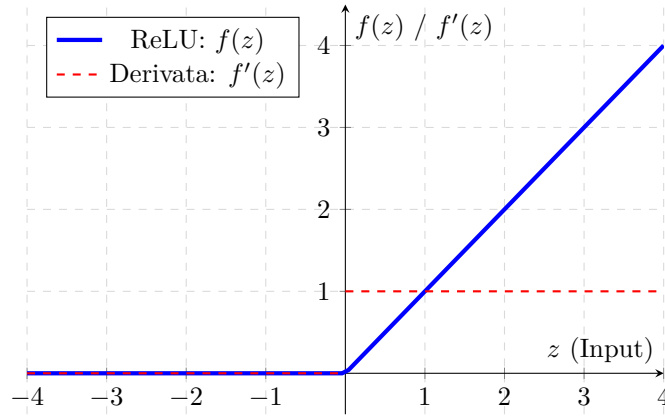


Figura 4: Rappresentazione geometrica della funzione ReLU (linea blu) e della sua derivata parziale (linea tratteggiata rossa).

dato che se $z < 0 \Rightarrow y = 0$, significa che il neurone è morto, e più layer ci sono più è probabile esponenzialmente che altri neuroni muoiano (inutili al riconoscimento).

8.2 exploding gradient:

Nonostante la ReLU abbia una derivata pari a 1 (per $z > 0$), la rete rimane comunque soggetta all'**Exploding Gradient**. Questo accade perché la *Chain Rule* moltiplica anche i **pesi sinaptici** (w) degli strati successivi:

$$\frac{\partial \text{Loss}}{\partial w_1} = \frac{\partial \text{Loss}}{\partial y_4} \cdot \underbrace{\frac{\partial y_4}{\partial z_4}}_1 \cdot \underbrace{\frac{\partial z_4}{\partial y_3}}_{w_4} \cdot \underbrace{\frac{\partial y_3}{\partial z_3}}_1 \cdot \underbrace{\frac{\partial z_3}{\partial y_2}}_{w_3} \cdot \underbrace{\frac{\partial y_2}{\partial z_2}}_1 \cdot \underbrace{\frac{\partial z_2}{\partial y_1}}_{w_2} \cdot \underbrace{\frac{\partial y_1}{\partial z_1}}_1 \cdot \frac{\partial z_1}{\partial w_1}$$

Isolando i passaggi tra i livelli, il gradiente finale risulta direttamente proporzionale al prodotto geometrico dei pesi:

$$\text{Gradiente} \propto \text{Loss} \cdot 1 \cdot w_4 \cdot 1 \cdot w_3 \cdot 1 \cdot w_2 \cdot 1$$

$$\text{Gradiente} \propto \text{Loss} \cdot (w_4 \cdot w_3 \cdot w_2)$$

Conclusione: Se i pesi intermedi assumono valori anche leggermente superiori a 1 (es. $|w| > 1$), il loro prodotto continuativo cresce in modo esponenziale con la profondità, generando aggiornamenti mastodontici e distruttivi. Per risolvere questo specifico problema si applica il **Gradient Clipping**, che forza il gradiente a rientrare in un intervallo controllato se supera una soglia limite.

9 skip connection (res net)

invece di calcolare $F(x) = H(x) - x$ (differenza tra output ideale e output della rete),
calcoliamo $H(x) = F(x) + x$
dove:

- $H(x)$ = output ideale (reale)
- $F(x)$ = residuo, quanto manca all'output ottimale
- x = output della rete

Questo è utile per evitare il gradient vanishing e per capire meglio come abbiamo rigirato la formula, possiamo vedere $F(x)$ come un'immagine dopo delle convoluzioni - l'immagine iniziale (il Δ cambiamento). $H(x)$ come l'immagine ottimale trasformata dai pixel.

nella back propagation:

$$\boxed{\frac{\partial \text{loss}}{\partial x} = \frac{\partial \text{loss}}{\partial H(x)} \cdot \left(\frac{\partial F(x)}{\partial x} + 1 \right)} \longrightarrow \text{derivato da } \overbrace{F(x) + x}^{H(x)} \Rightarrow \partial F(x) + 1$$

questo +1 ci permette di non rendere il gradiente troppo piccolo e quindi di non perdere l'informazione iniziale (l'immagine iniziale).

quindi se:

- $H(x) - x = 0 \rightarrow F(x) = 0$ (la rete apprende a non fare nulla) (*input vicino all'immagine ideale*)
- $H(x) - x \neq 0 \rightarrow F(x) \neq 0$ (la rete apprende per fare meglio)

grazie al clipping (applicato ad ogni strato) si rimuove il problema della rete profonda (vanishing gradient).

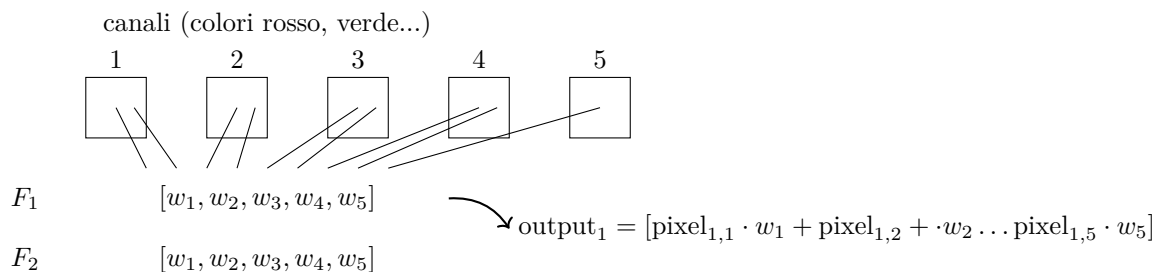
10 Possibili problemi che possono sorgere:

10.1 riduzione canali

abbiamo visto come da un singolo canale, applicando filtri in parallelo possiamo aumentarli, ma se ho 64 canali e voglio ridurli a 32? Per ridurli uso i filtri 1×1 .

in questo caso: 32 filtri (output) con 64 pesi ciascuno (1 per ogni canale).

in pratica si fa una sorta di mix tra i vari strati pesandoli tra loro.



Serve a trovare le caratteristiche comportamentali, ad esempio un nuovo canale descriverà il rosso e i bordi, più nel dettaglio, ogni nuovo filtro conterrà le informazioni di tutti e 64 (dato che ogni suo pixel è la media pesata di tutti i corrispettivi pixel dei 64 filtri), sarà durante l'addestramento che verranno scelti in automatico i pesi di ogni nuovo filtro per selezionare i canali più rilevanti.

10.2 problema rotazioni

se si fornisce un oggetto ruotato la rete può non riconoscerlo dato che la rete spesso ragiona in base alla posizione dei pixel.

soluzione:

per ogni immagine di training, fornire la stessa ruotata in varie posizioni.

10.3 reti che svolgono più compiti

le reti neurali possono essere addestrate a fare più cose insieme. Ad esempio riconoscere un'immagine e riconoscere la sua posizione (box sul fiore o sul cane). Come si può fare se la rete è addestrata per imparare un singolo compito?

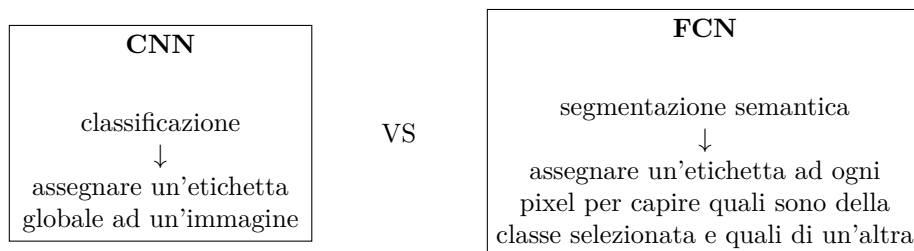
soluzione:

1. tra gli strati di **convoluzione** e gli strati densi (**fully connected**) di classificazione, esiste uno strato **flatten** il quale appiattisce la matrice in un vettore.
2. successivamente il vettore viene sdoppiato e viene fatto passare negli strati fully connected ciascuno con il tipo di previsione (classe, posizione...).

le rispettive loss function vengono sommate. infatti le loss si condividono poiché i pesi della rete convuluzionale (non le reti sdoppiate fully connected) vengono usati per entrambi i compiti.

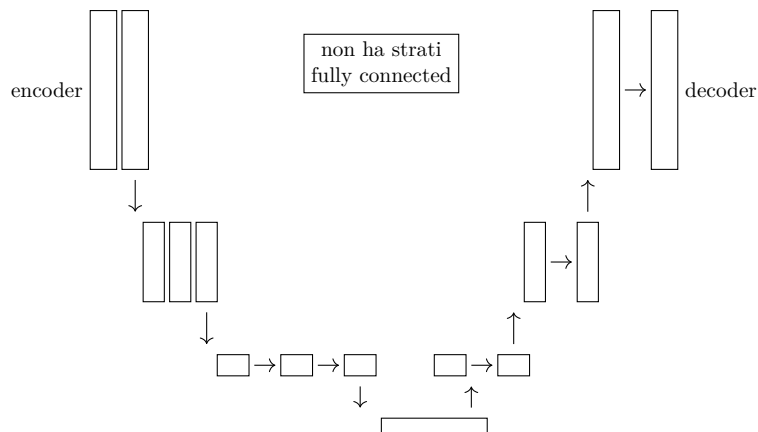
11 cnn vs fcnn e u-net

Ci sono 2 approcci diversi per l'identificazione dell'immagine:



11.1 u-net

Le **U-net** hanno lo scopo di segmentare un'immagin. Prendono un'immagine in input e ne restituiscono una con lo stesso numero di pixel ma segmentata in output.



1. **encoder:** classico strato convoluzionale con pooling.

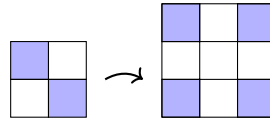
2. **decoder:** un pooling di 2 tipi:

- interpolation
- transposed convolution / up convolution

nel decoder si usa la **concatenazione** (concateno sotto l'immagine elaborata, l'immagine non elaborata) per riconoscere dettagli spaziali persi es: so che c'è una macchina in basso ma non so dove con precisione.

11.1.1 interpolation

non ha pesi, non impara, semplice



- **nearest neighbor** →

1	2
3	4

 ⇒

1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4
- **bilinear** → media pesata dei 4 pixel vicini
- **bicubic** → usa 16 pixel vicini e fa un'interpolazione più complessa

11.1.2 transposed convolution / up convolution

reversed pooling (unpooling) + convolution

- si ingrandisce l'immagine inserendo zeri prima e si fa il padding per il filtro che si userà
- si usa un filtro (es: 2x2) e viene passato sull'immagine per sostituire gli zeri con dei valori (il filtro ha i suoi pesi)

11.2 allenamento u-net

- per ogni pixel avremo un vettore one-hot $[0, 0, 1, 0]$ che indica la vera classe del pixel (per esempio a trasformazione avvenuta, tutti i pixel della macchina saranno simili).

inizializzazione dei pesi:

- glorot/xavier initialization: (sigmoide, tanh)

limita la varianza del gradiente

uniform

$$\text{limit} = \sqrt{\frac{2 \cdot c}{\text{fan_in} + \text{fan_out}}}$$

rende più variabili i pesi ma sempre limitandoli

normal

$$\sigma = \sqrt{\frac{2 \cdot c}{\text{fan_in} + \text{fan_out}}}$$

- $\text{fan_in} = n^\circ$ input neurons
- $\text{fan_out} = n^\circ$ output neurons
- c = parametro scelto
- limit = limite scelta pesi $[-\text{limit}, \text{limit}]$
- σ = limite scelta pesi $[0, \sigma^2]$

- **he initialization:** (relu)

stesso ragionamento:

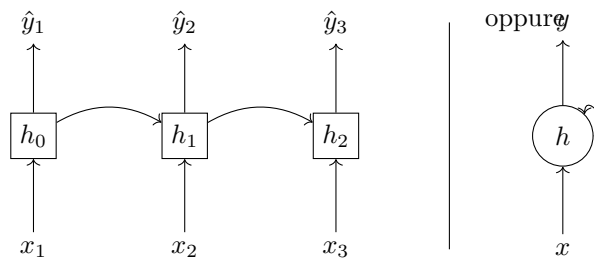
$$\boxed{\text{uniform}} \\ \boxed{\text{limit} = \sqrt{\frac{2 \cdot c}{\text{fan_in}}} \quad [-\text{limit}, \text{limit}]}$$

$$\boxed{\text{normal}} \\ \boxed{\sigma = \sqrt{\frac{2 \cdot c}{\text{fan_in}}} \quad [0, \sigma^2]}$$

12 rnn (recurrent neural network)

Sono reti usate per la comprensione del testo dove l'input e l'output è determinato sia dall'input di ora che da quello precedente.

12.1 struttura:



il concetto è che se prima ogni layer aveva i suoi pesi, ora i layer h condividono gli stessi pesi in modo da "decidere" in funzione del passato.

12.2 Architettura e Logica delle RNN Classiche

Le reti neurali ricorrenti (*Recurrent Neural Networks*, RNN) nascono per elaborare dati sequenziali in cui l'ordine temporale degli elementi è informativo (es. testo o serie storiche). A differenza delle reti *Feed-Forward*, le RNN possiedono un ciclo interno che consente all'informazione di persistere.

La logica della rete si basa sul concetto di **stato nascosto** (*hidden state*) h_t , che agisce come una memoria dinamica a breve termine. Ad ogni passo temporale t , la rete riceve l'input corrente x_t e lo combina con la memoria del passato immediato h_{t-1} mediante la seguente equazione di transizione:

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b) \quad (9)$$

Il nesso ingegneristico fondamentale risiede nella **condivisione dei parametri**: le matrici dei pesi W_A (per la memoria) e W_x (per l'input), insieme al vettore di bias b , sono **identiche** per ogni passo della sequenza. La rete non crea nuovi strati per ogni elemento, ma riutilizza ciclicamente la stessa struttura algebrica, permettendo al sistema di generalizzare la comprensione del contesto indipendentemente dalla lunghezza dell'input.

12.3 bptt: (back propagation through time)

stesso principio della bp con qualche piccola differenza.

concetto:

	x_1	x_2	x_3	
input:	i	love	cat	
previsione:	io	amo	i	gatti
	y_1	y_2	y_3	$\dots$

- calcolo la loss per ogni output:

$\hat{y}_1$	$\hat{y}_2$	$\hat{y}_3$	$\text{loss}(\hat{y}_1, y_1)$
$\uparrow$	$\uparrow$	$\uparrow$	
h_1	$\longrightarrow h_2$	$\longrightarrow h_3$	$\text{loss}(\hat{y}_2, y_2)$
$\uparrow$	$\uparrow$	$\uparrow$	
x_1	x_2	x_3	$\text{loss}(\hat{y}_3, y_3)$

- calcolo la variazione di ogni peso di h e aggiorniamo il piano:

all'indietro: (in funzione dell' y successivo)

$$\begin{aligned}\Delta w_{h_3} &= \text{loss}(\hat{y}_3, y_3) \\ \Delta w_{h_2} &= \text{loss}(\hat{y}_2, y_2) + \text{loss}(\hat{y}_3, y_3) \\ \Delta w_{h_1} &= \text{loss}(\hat{y}_1, y_1) + \text{loss}(\hat{y}_2, y_2) + \text{loss}(\hat{y}_3, y_3)\end{aligned}$$

naturalmente calcolando la derivata della loss rispetto al peso.

13 vanishing gradient e exploding gradient

poiché c'è una ricorrenza nell'elaborare con la stessa matrice dei pesi memoria e input:

$$h_t = W_h \cdot \overbrace{h_{t-1}}^{\text{memoria}} + W_x \cdot \overbrace{x_t}^{\text{input}}$$

traslando l'input e vediamo che lo stiamo di fatto moltiplicando:

$$h_t = W_h \cdot (W_h \cdot (W_h \cdot h_{t-3})) = W_h^3 \cdot h_{t-3}$$

W_h^3 implica che se λ (autovalore di W) è:

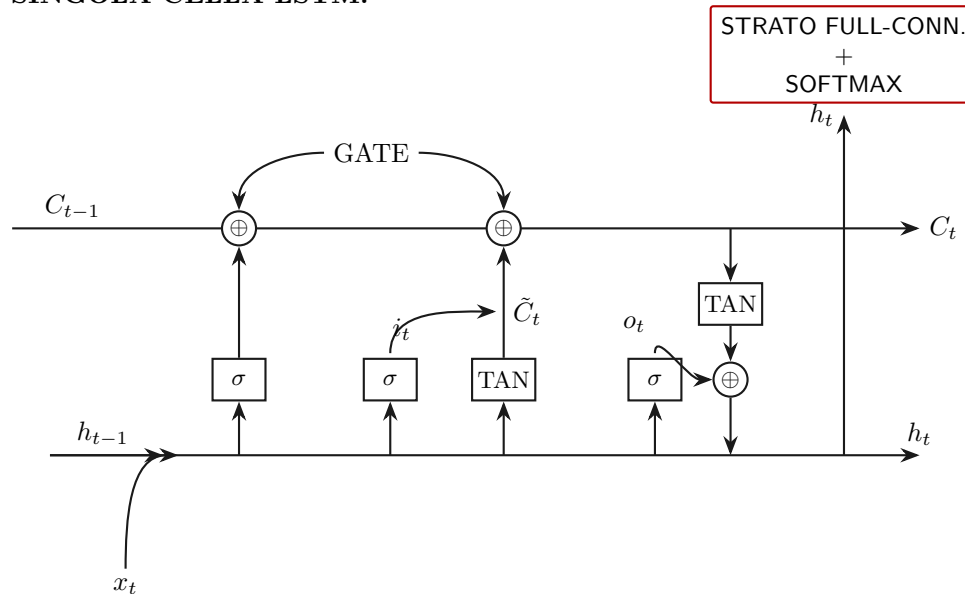
- $< 1 \rightarrow$ continuo diminuisco (vanishing)
- $> 1 \rightarrow$ continuo aumento (exploding)

questo problema si risolve cambiando la struttura dell'**RNN** creando la struttura LSTM.

14 lstm: (long short-term memory)

architettura di una rnn in grado di decidere per ogni cella quanta memoria dimenticare e quanto conservare il nuovo input per poi generare una previsione.

SINGOLA CELLA LSTM:



h_t **si biforca** perché uno va nella cella succ. e l'altro servirà per calcolare il gradiente e la loss

14.1 forget gate: (quanta memoria dimenticare)

$$f_t = \sigma \cdot (W_f \cdot [h_{t-1}, x_t] + b_f)$$

- f_t = output forget gate
- σ = sigmoide (tra 0 e 1) decide quanto dimenticare
- W_f = matrice pesi forget gate
- $[h_{t-1}, x_t]$ = concatenazione output cella precedente e input corrente
- b_f = bias forget gate

concateniamo anche con l'input per avere una decisione informata (in base a cosa viene deciso quanto dimenticare).

14.2 input gate: (quanta informazione conservare)

Stessa struttura della forget gate, varia solo il bias.

$$i_t = \sigma \cdot (W_i \cdot [h_{t-1}, x_t] + b_i)$$

- i_t = output input gate
- σ = sigmoide
- W_i = pesi input gate (come il forget gate, stessa struttura, varia solo il valore del bias e dei pesi)
- $[h_{t-1}, x_t]$ = concatenazione
- b_i = bias input gate

attenzione: i_t indica la nostra capacità critica, infatti i_t indica di quanto conservare l'input in funzione di ciò che sappiamo.

ma di cosa dobbiamo moltiplicare l'input? Il concetto consiste nel moltiplicare la nostra capacità critica per l'informazione nuova, cioè $\tilde{C}_t$

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

- unica differenza è che si usa la tangente (range -1, 1) in modo da modulare anche in negativo. Questo è importante perché la memoria a lungo termine (C_t) cresce per accumulo, avendo $\tilde{C}_t$ anche negativo, ci permette di modificarla non solo in crescita.

ora quindi la nuova memoria sarà:

$$C_t = \underbrace{f_t}_{\text{forget gate}} \cdot \underbrace{C_{t-1}}_{\text{memoria a lungo termine precedente}} + \underbrace{i_t \cdot \tilde{C}_t}_{\text{nuova informazione}}$$

14.3 output gate:

Calcola quanto conservare dell'output gate precedente in base all'input per poi moltiplicare per la memoria di ora

- capacità razionale:
$$o_t = \sigma \cdot (W_o \cdot [h_{t-1}, x_t] + b_o)$$
- quanto e cosa mostrare:
$$\tilde{h}_t = \tanh(C_t) \rightarrow C_t \text{ memoria a lungo termine}$$
- output effettivo:
$$h_t = o_t \cdot \tilde{h}_t$$

ci sono 2 strade di previsioni nella previsione:

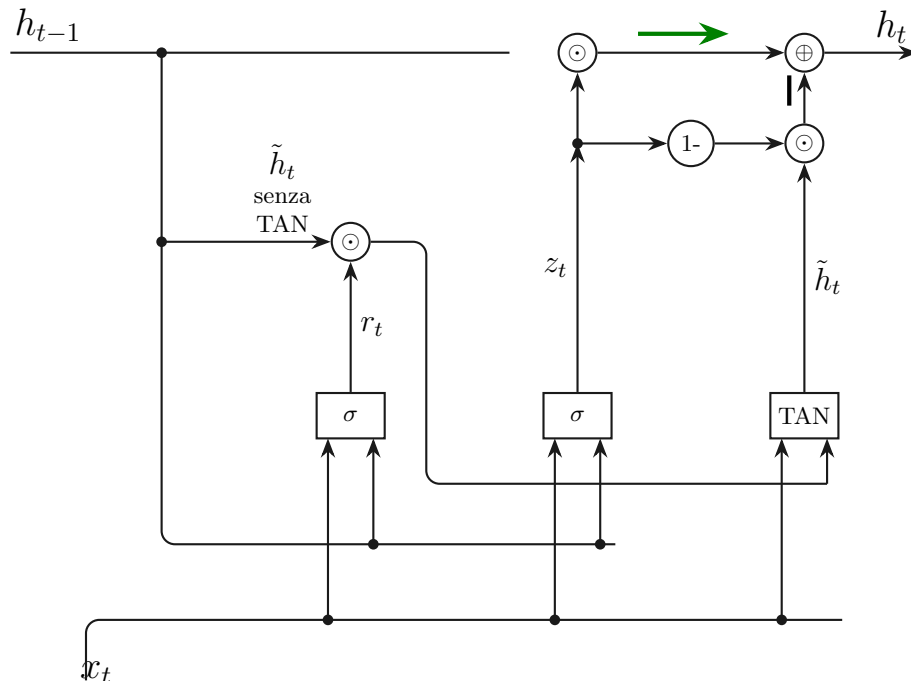
- **prevedere ogni lettera:**
 - 26 lettere + 100 simboli
 - ignoranza
 - usate in correzione testo / generazione
- **ogni parola:**
 - 50k parole
 - costosa
 - usata in chat bot

15 gru

Le reti **GRU** nascono per risolvere la complessità delle **LSTM**, infatti nonostante risolvano il problema del gradiente introducendo la memoria a lungo termine, risultano ancora computazionalmente onerose.

La **GRU** usa solo 2 gate:

- reset
- update



15.1 reset gate:

serve a capire quanto è rilevante ciò che è appena successo per la previsione futura, **MEMORIA A BREVE TERMINE**

$$r_t = \sigma \cdot (W_r \cdot x_t + U_r \cdot h_{t-1})$$

- h_{t-1} = stato precedente
- x_t = input corrente
- W_r, U_r = pesi reset gate
- σ = func sigmoide (0,1)

poi calcola il nuovo stato:

$$\tilde{h}_t = \tanh(W_h \cdot x_t + U_h \cdot (r_t \odot h_{t-1}))$$

(dove $\odot$ indica il prodotto elemento per elemento).

ATTENZIONE: nonostante le formule siano molto simili, il fattore determinante sono le matrici dei pesi, le quali specializzano la formula ad un determinato compito

15.2 update gate:

controlla quanto dello stato precedente h_{t-1} tenere e quanto del nuovo stato $\tilde{h}_t$ usare (**MEMORIA A LUNGO TERMINE**):

$$z_t = \sigma \cdot (W_z \cdot x_t + U_z \cdot h_{t-1})$$

- x_t = input
- h_{t-1} = stato prec.
- W_z, U_z = pesi update func
- σ = sigmoide (0,1)

↓

agisce come peso tra output del reset e stato precedente.

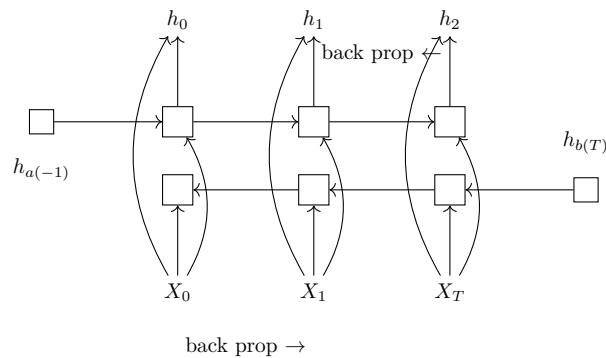
15.3 output:

calcola lo stato finale grazie a z_t il quale lega reset gate per i concetti immediatamente presenti e i concetti passati per dare contesto:

$$h_t = \underbrace{(1 - z_t) \odot \tilde{h}_t}_{\text{Il Presente}} + \underbrace{z_t \odot h_{t-1}}_{\text{Il Passato}}$$

16 bidirectional rnn

Le reti bidirezionali sono utili quando per capire il significato di una parola dobbiamo per forza guardare nel futuro. Infatti nelle reti che abbiamo visto fino ad ora, la loro struttura le vincola a conoscere solo al passato.



h_0 combinazione di $h_{a(0)}$ e $h_{b(0)}$, infatti tutti gli h sono precalcolati, solo successivamente combinati come appena detto per creare il vettore h finale da usare per la predizione.

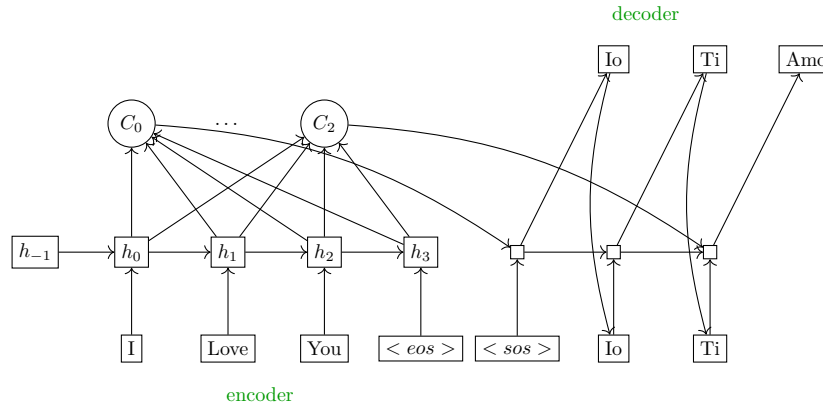
$$h_{(t)} = [h_{a(t)}; h_{b(t)}]$$

la backprop funziona come le normali rnn ma abbiamo 2 back prop una per una direzione (da fine a inizio) e una per l'altra (da inizio a fine).

17 seq2seq con attenzione

modelli utili per la traduzione, sono asincroni, prima viene processato tutto l'input poi viene processato l'output. In questi modelli introduciamo anche il concetto di **attenzione** (C_t). l'attenzione è un vettore che serve a dare il contesto necessario basato su tutti gli h_t (i quali rappresentano i token singoli elaborando però il contesto fino a lì).

17.1 struttura:



$$c_t = \frac{1}{N} \cdot \sum_i^N w_i(t) \cdot h_i$$

17.2 passaggi:

1. calcolo ciascun h_t di ogni token prendendo in pasto ogni volta h_{t-1} .
2. calcolo per ogni token C_t che è un vettore di pesi (uno per ogni h_t), raffigura l'attenzione, cioè l'informazione utile per ogni token in una determinata posizione in base al contesto che lo circonda (per questo per calcolarla si prende in pasto tutti gli h_t).
3. finita la fase di encoding, si passa nel decoder dove la logica consiste nel pesare ogni h_t in base alla C_t corrispondente
4. oltre a passare gli h_t pesati, si passa al modello anche la parola in output appena emessa nel ciclo precedente (il suo embedding), questo perchè serve al modello come segnalibro, ovvero capire dove si trova nel testo e che regole grammaticali usare.
5. per creare effettivamente una parola in output dobbiamo creare un vettore s_t (generalmente come in **GRU**) e usare la formula:

$$\text{Logits} = W_o \cdot s_t + b$$

la quale creerà un enorme vettore con le probabilità per ogni vocabolo.

17.3 Back propagation:

1. **Calcolo dell'Errore Istantaneo ($\mathcal{L}_t$):** Il processo si innesca al termine del passo di decodifica t . Viene calcolata la funzione di costo (*Cross-Entropy Loss*) tra la distribuzione stocastica emessa dalla Softmax e il token target reale del dataset.
2. **Retropropagazione attraverso la Testa di Output:** Il gradiente dell'errore risale lo strato di normalizzazione Softmax, determinando l'errore sui punteggi grezzi (*logits*). Successivamente, attraversa la matrice di proiezione lineare finale W_o (matrice dei pesi per calcolare s_t), giungendo sullo hidden state corrente del decoder:

$$\frac{\partial \mathcal{L}_t}{\partial s_t} = W_o^T \cdot \frac{\partial \mathcal{L}_t}{\partial z_t} \quad (10)$$

3. **Biforcazione Interna alla Cella del Decoder:** Lo stato s_t è il prodotto della sintesi operata dalla cella ricorrente (es. GRU o LSTM). Entrando nella cella, il gradiente si scinde lungo tre direttrici di derivazione parziale:
 - Verso lo hidden state precedente s_{t-1} , per propagarsi a ritroso lungo la catena temporale del decoder.
 - Verso l'embedding del token precedentemente emesso $\hat{y}_{t-1}$, per aggiornare i pesi del dizionario target.
 - Verso il vettore contesto c_t , che costituisce il punto di giunzione con l'encoder.
4. **Attraversamento e Scissione sul Nodo di Attenzione (c_t):** Il gradiente giunto sul vettore contesto ($\frac{\partial \mathcal{L}_t}{\partial c_t}$) incontra l'operatore di combinazione lineare dell'attenzione. Per la regola della catena (*chain rule*), l'errore si divide simultaneamente in due flussi:
 - **Ottimizzazione dell'Allineamento:** Il gradiente fluisce verso i pesi stocastici $w_i(t)$, risalendo la Softmax del modulo di attenzione e la funzione di *score*. Questo permette di correggere le matrici responsabili della dinamica di focalizzazione (il “mirino” del modello).
 - **Iniezione nell'Encoder:** Il gradiente attraversa linearmente il canale di comunicazione, riscalato dal rispettivo peso di attenzione, iniettandosi direttamente negli hidden state dell'encoder:

$$\frac{\partial \mathcal{L}_t}{\partial h_i} = w_i(t) \cdot \frac{\partial \mathcal{L}_t}{\partial c_t} \quad (11)$$

5. **Accumulazione dei Gradienti sull'Encoder:** Poiché ciascuno stato dell'encoder h_i può aver contribuito alla generazione di molteplici token del decoder, il gradiente totale che sollecita l' i -esimo nodo è la sommatoria dei contributi derivanti da tutti i passi temporali U del processo di decodifica:

$$\nabla_{h_i} \mathcal{L} = \sum_{t=1}^U \frac{\partial \mathcal{L}_t}{\partial h_i} \quad (12)$$

6. **Discesa Ricorrente lungo la Catena dell'Encoder:** Una volta accumulato il gradiente complessivo su ciascun h_i , l'algoritmo esegue la classica retropropagazione temporale all'indietro lungo l'asse dell'encoder ($i = T \rightarrow 1$). Questo flusso finale consente l'aggiornamento dei parametri sinaptici interni ($W_{\text{enc}}, U_{\text{enc}}$) e delle matrici di embedding associate alla sequenza di input originale x_i .